VELDORA STUDIO

Development Proposal

Weather Monitoring and Alerting System — Sydney Metro M1 Line

In response to the Technical Proposal for Weather Monitoring, Sydney Metro M1 (OBS-MTS-M1-WX-2026-01, Rev A, 22 July 2026), revised following Observator's feedback of 3 September 2026

Submitted by	Veldora Studio
Contact	Hassan Ali · hassanali@veldorastudio.com
Prepared for	Observator Instruments (Observator Group)
Scope	Ingestion pipeline, central server and web portal
Duration	3 months from kickoff, with a release every month
Proposal reference	VS-OBS-MTS-M1-2026-02
Version / date	Rev B · 3 September 2026
Status	Commercial-in-confidence

Contents

1. What has changed in this revision

Rev A proposed ten months, a native mobile application, SMS delivery, and a platform designed for one-second data. Following your feedback we have revised all four, and simplified the infrastructure that those choices had required.

The scope of what the system does is unchanged — every requirement in the Statement of Requirements is still delivered — but the way it is built and the time it takes are substantially reduced.

	Rev A	Rev B
Duration	10 months	**3 months, a release every month**
Mobile	Native iOS/Android app (Flutter)	**Responsive web display — no separate app**
Notifications	Screen, email, SMS	**Screen, email, web push**
Data rate	Designed for 1 second	**1-minute logging, batched transmission**
Yearly readings	~250 million	**~4.2 million**
Infrastructure	Lightsail, SQS, MongoDB, Redis	**Lightsail, MongoDB**

Each of these removes work. Dropping the native application removes an entire codebase, two app-store submissions and a separate release cycle. Replacing SMS with web push removes provider procurement, per-message cost and the delivery-receipt plumbing that went with it. Moving from one-second to one-minute data reduces the volume by a factor of sixty, which takes the storage and query design from a hard problem to an ordinary one.

The last row follows from the one above it. A message queue and a cache were in Rev A because one-second data needed them. At one-minute they earn nothing, and §5 and §7 set out what does the same job instead. Fewer moving parts is not only faster to build — it is less to secure, less to monitor and less to hand over.

2. What we are building

A complete monitoring and alerting platform for seven trackside weather stations on the Sydney Metro M1 line, covering rainfall, flood water level, temperature, humidity and wind — plus supervised control of the trackside flood pumps at Marrickville.

This is a **safety-critical rail system**. Its alerts trigger front-of-train patrols, temporary speed restrictions, and blocking of the line. The engineering standard throughout is that **no reading is ever lost and no alert is ever missed**, and that every design decision can be explained in those terms.

The three deliverables

#	Deliverable	Technology
1	**Ingestion pipeline** — from the trackside logger to the database, losing nothing	Secure agent on AWS Lightsail → NestJS
2	**Central server** — storage, alert engine, notifications, API	**NestJS** (TypeScript), MongoDB
3	**Web portal** — map, trends, station and pump screens, on desktop and phone	**Next.js** (TypeScript), responsive

There is no fourth deliverable in this revision. The mobile experience is the same application, laid out for a phone — see §9.

3. The seven stations

Each station carries only the sensors its location requires.

#	Location	Rain	Water level	Float switch	Temp / Humidity	Wind	Pumps
1	Marrickville	√	√	√	–	–	√
2	Marrickville–Dulwich Hill	–	√	√	√	√	–
3	Canterbury	–	√	√	–	–	–
4	Campsie	–	√	√	–	–	–
5	Belmore triangle	√	√	√	√	√	–
6	Lady Game Drive (tunnel)	–	√ ×2	√ ×2	–	–	–
7	Windsor Road SSC	–	–	–	√	√	–

Location 6 is two independent units — up tunnel and down tunnel — each with its own logger. The platform therefore handles **eight loggers across seven locations**, presenting Location 6 as one place with two monitoring points.

4. Data rate and retention

Per your specification, the OMC-048 **logs 1-minute data** and transmits over 4G/5G **in batches, by default every five minutes and configurable**, with an **immediate transmission on any threshold breach**. That last part matters and §10 returns to it: it is what keeps a breach inside the five-minute alert obligation regardless of the batch interval.

Loggers	8
Readings per logger	1 per minute
Per day	**11,520**
Per month	**~350,000**
Per year	**~4.2 million**

This is a comfortable volume for a single database — roughly one sixtieth of what Rev A was designed to carry. Two useful consequences follow.

Retention stops being a difficult decision. At 4.2 million readings a year, keeping full detail for the entire life of the contract is inexpensive. Rev A offered a choice between one month, six months and a year of detail because one-second data made that choice matter financially. It no longer does. We propose keeping **full detail for the contract term and aggregates permanently**, and we will confirm this with you in Month 1 rather than assume it.

The storage design gets simpler, but we keep the parts that pay for themselves. Per-minute, per-hour and per-day rollups are still computed as data arrives, because a twelve-month trend should read a few hundred daily rows rather than four million raw ones. That is cheap to build and it is what keeps the portal fast in year three.

Where real-time control actually lives

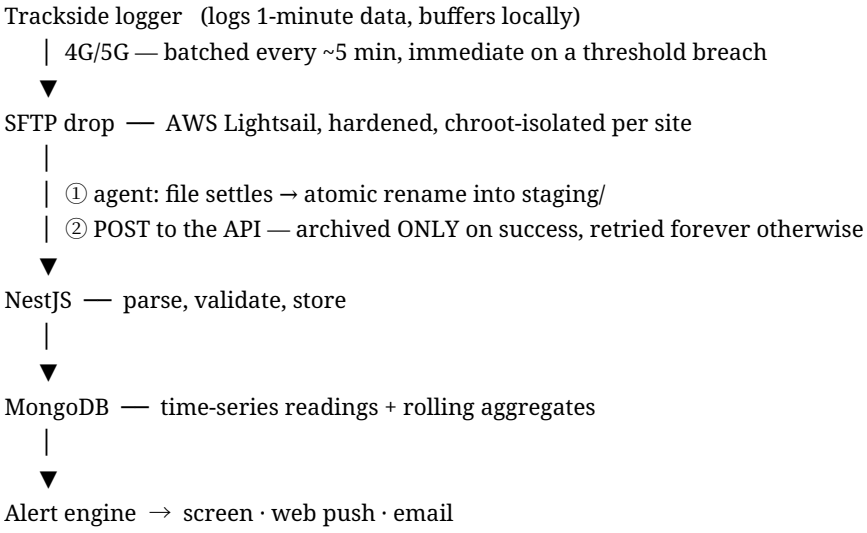
This is the reason a batched server interval is safe.

The pump start/stop decision runs **on the OMC-048 logger at the station**, in real time, independent of the network. That is correct and we will not move it: a pump that stops working because a cellular link dropped is not acceptable on a flood site. Your own specification makes the same point — pumping and local alarming continue through any loss of the central server or cellular link, with buffered data forwarded on restoration.

The central server's job is to **aggregate, alert, display, record and supervise** — and to accept a manual override from an authorised operator. None of those require second-by-second data.

5. Ingestion — how nothing is lost

The obligation is that a failure anywhere **delays** data rather than **losing** it. Rev A met it with a message queue. This revision meets it with three independent buffers that already exist, none of which has to be built or operated as a separate service.



The three buffers

If this is unavailable	What holds the data	For how long
Cellular, or the whole Lightsail box	The **logger's own** store-and-forward buffer, per your specification	Per the logger's local storage
The ingest agent	Files accumulate in `upload/` on disk	Until disk is exhausted
The API or the database	Files wait in `staging/` on disk, retried with backoff	Until disk is exhausted

The ordering is what makes this safe. The agent renames a file into `staging/` **before** it posts, and moves it to `archive/` **only** once the server confirms the write. A crash between the two leaves the file staged, so it is retried rather than lost. A file is never deleted at any point.

At roughly four megabytes a day of small CSV files, a month-long outage occupies about 120 MB. For comparison, a managed queue would have capped retention at fourteen days; the disk does not.

Guarantees we build in

Guarantee	How
No duplicates	Every file carries a content hash; a repeat is recognised and skipped, even if a success response was lost in transit
No loss on crash	Rename-into-staging before sending; archived only once the server confirms
No loss on outage	Files accumulate and are replayed automatically, oldest first, on reconnection
Nothing deleted	Ingested files are archived permanently, never removed
Out-of-order safe	Readings are keyed by their own timestamp, not arrival order
Poison data cannot block	An unparseable file is quarantined and reported; everything behind it keeps flowing
Auditable	Every file's journey is recorded — received, parsed, stored, archived

What this does not cover, and how we cover it

No buffer delivers **alerts** while the server is down — a queue would not have either. Alerting depends on the server running, so that is treated as an availability problem: health checks, a documented restore rehearsal, and a monitored "station silent for more than 15 minutes" condition. The one failure worth naming is that a station quietly retrying looks exactly like calm weather, so that alert is raised on silence rather than on error.

6. Security

Data security is treated as a requirement of the same rank as correctness, and is unchanged from Rev A.

In transit

- **SFTP only** from the trackside box, on hardened OpenSSH with key-based authentication and per-site chroot isolation, so one site can never see another's data.
- **TLS everywhere else** — logger to server, server to database, browser to API.
- The ingest agent holds an **outbound-only credential**. The trackside box exposes **no inbound port** to us.

At rest and in the platform

- **Encryption at rest** on the database and on backups.
- **Least-privilege credentials** — the ingest credential can upload readings and nothing else. It cannot read other sites' data, change configuration, or administer anything.
- **Every secret hashed**, never stored recoverably. Generated passwords are shown once.
- **Role-based access control** with per-station scoping, so a user sees only what their role and their sites permit.
- **Optional multi-factor authentication** for portal access.
- **Full audit trail** — every login, threshold change, acknowledgement and manual pump command recorded against a named person with a timestamp.
- **Automated dependency scanning** in the build, and a **penetration test** before go-live.

A smaller stack is a smaller attack surface. Removing the queue and the cache removes two network services, two sets of credentials and two components to patch.

Scripts on the Lightsail box

Everything privileged is a single root-owned script with a single narrow sudo rule. Arguments are validated in three independent layers, so no one check is load-bearing. The agent runs unprivileged, cannot modify its own code, and can execute exactly one command as root — nothing else.

7. Performance

The requirement is that years of history stay fast to query and fast to display. At 4.2 million readings a year this is straightforward, and these are the decisions that keep it that way.

- **Time-series collections**, purpose-built for this shape of data — high write rate, append-only, queried by time range.
- **Compound indexes** matched to the queries that actually run, verified with query plans rather than assumed.
- **Pre-aggregated rollups.** Every reading is summarised into per-minute, per-hour and per-day records as it arrives, so a twelve-month chart reads a few hundred rows.

- **Live values held in the API process** and pushed to the browser over WebSocket. With eight stations this is a few kilobytes of state; the authoritative copy is a single indexed lookup per station, which is why a separate cache service earns nothing here.
- **Cursor pagination** everywhere, and **streamed CSV export**, so a large query or a long export cannot exhaust server memory.
- On the phone: server-side rendering, code splitting and downsampled charts, so the first screen is fast on a field connection.
- A **performance budget enforced in the build**, so a regression fails the pipeline rather than reaching production.

The portal runs as a single API instance behind TLS, which is ample at this volume. Should it later be scaled to several instances, live fan-out moves to MongoDB change streams — a configuration change, not a redesign.

8. The alert engine

The heart of the system, and the part with the least tolerance for error. This section is unchanged from Rev A — the logic is a requirement, not an implementation choice.

Rainfall — vigilance logic

For each threshold — **≥25 mm/hr**, **≥45 mm/3 h**, **≥120 mm/3 days** — the engine maintains a live rolling total and compares it continuously.

When a threshold is crossed it raises the alert immediately with the exact MTS wording. When rainfall falls back below the line it does **not** stand down: it starts a **vigilance countdown** — **6 h / 12 h / 48 h** respectively — and holds the alert for the whole period. Further qualifying rain **restarts** the countdown. When the clock finally expires it issues an automatic **all-clear**.

Every value — thresholds, windows, countdown durations, wording — is a configuration item MTS can change without a software release.

Flood, temperature and wind

Parameter	Logic
Flood	Standing water (PTZ verification prompt) → water above rail foot (block the line) → trending down (staged reinstatement 25 → 60 kph → unrestricted)
Temperature	≥38 °C and ≥45 °C rising, sustained >5 min; falling <45 °C / <38 °C sustained >10 min; drives the 60/40 kph TSR and heat-patrol wording
Wind	≥75 / ≥85 kph TSR wording at the listed kilometrages; ≥120 kph block-line on all external track
Pumps	Start, stop, fail, no-flow, high-high level, sensor discrepancy — all raised immediately
System	Sensor offline, missing data, low battery, communications loss — leading indicators, not just failures

Dwell timers of five and ten minutes are unaffected by 1-minute logging: a five-minute sustained condition is evidenced by five consecutive readings rather than three hundred.

Redundancy and fallback

On sensor failure the engine automatically generates alerts from the **designated alternate source** and **rewords** the alert to state which additional sections it now covers. A fault alert is issued first, telling MTS that alerting has moved to the fallback — so nobody is unaware that they are running degraded.

9. The web portal, on desktop and phone

One responsive application — phone, tablet, laptop — with no separate mobile site and no separate application to maintain.

Screen	Contents
Corridor map	Every station on a map, live readings shown against their thresholds, colour-coded status, system-health summary and a station-status table. Click a site for its detail — the top-level view MTS asked for
Station detail	All sensors for that location, live and historical, with thresholds drawn on the charts
Trends	Time-series for wind (mean and gust), rainfall intensity, temperature, humidity and level, with threshold lines and flexible date ranges
Flood event	Water-level series for adjacent locations, pump auto-start markers, pump status, affected-vicinity inset, timestamped event log
Marrickville pump station	Live sensors, level trend against pump-start and high-high lines, duty/standby status and condition, **AUTO/MANUAL toggle**, **supervised START/STOP** with confirmation
System health	Refreshed at ≤10 minutes: inoperable, unresponsive, missing sensor or pump data, low battery and other leading indicators
Alerts & notifications	Full searchable history, filter by severity / acknowledgement / location / date, unacknowledged badge, per-event acknowledge, CSV export
Historical query	Point-and-click by location, sensor and date range with optional averaging — no technical knowledge, no SQL. Paged sortable results, one-click CSV export
Administration	Users, roles, station scoping, thresholds, alert wording, recipients

On a phone

The phone layout is deliberately **simple and straightforward, with minimal interaction** — it is what an on-call person needs at two in the morning, not the full operations console:

- the alert that woke them, in full, with a link straight to the event;
- acknowledgement, recorded against their name;
- the corridor map and the live readings for any station;
- supervised pump control for authorised roles, with confirmation.

Everything else — administration, bulk history, reporting — remains available but is designed for a desk.

Users and roles

Twenty or more named logins, each individually authenticated, with **no shared accounts**. Rather than granting permissions one by one, the administrator assigns each user one or more **roles**; each role bundles a defined set of permissions following least privilege. A user may hold more than one, and every user's access is **scoped to specific stations or areas**, so people see only what is relevant to them.

The six standard roles below are provided, can be tailored, and **additional custom roles can be created** to match the MTS operating model.

Role	Typical user	Key permissions
Administrator	System owner / IT	Full control: manage users and roles, configure thresholds and alert recipients, and all view and control functions
Operator	OCC / control room	View all live dashboards and history, receive and acknowledge alerts, and initiate operational responses; cannot manage users or change configuration
Pump Controller	Authorised operations staff	All Operator permissions, plus supervised manual pump start/stop (MANUAL mode with confirmation) at permitted stations

Role	Typical user	Key permissions
Maintainer / Technician	Field and maintenance	Device and power health, calibration and fault flags, and maintenance mode; acknowledge maintenance alerts; limited configuration
Analyst / Reporting	Engineering and reporting	Historical query engine, trend analysis and CSV export; no live control
Viewer	Stakeholders	View dashboards and history only; no actions or changes

Administering them

MTS manages all of this from the administration console, with no dependency on us for day-to-day account changes:

- **Add a user** — name, contact details, one or more roles, and the stations or areas they may access.
- **Edit a user** — roles, station scope, contact details and notification preferences, changeable at any time.
- **Suspend or remove** — access is revoked instantly. A suspended account is blocked from signing in **but keeps its audit history**, so somebody leaving never erases the record of what they did.
- **Scale** — 20 or more concurrent logins, with no per-seat licensing constraint.

Accountability

Because every account is named, every action is attributable to a specific person.

- **Named accountability.** No shared logins; every login and significant action is tied to a named user.
- **Full audit trail.** User additions and removals, role changes, threshold and configuration edits, alert acknowledgements and manual pump start/stop — each recorded with user, timestamp and detail, and both queryable and exportable.
- **Separation of duties and least privilege.** Sensitive actions such as manual pump control are restricted to authorised roles and require MANUAL mode plus confirmation.
- **Session and password security.** Enforced password policy and account lockout, encrypted sessions, automatic inactivity timeout, and a **permission re-check on every action** rather than only at sign-in — so a role withdrawn takes effect immediately, not at the user's next login.

10. Notifications — how an alert reaches a person

Three channels, all built in Month 2.

Channel	Behaviour
On screen	Live over WebSocket — pop-up, unacknowledged badge, audible option for control-room use
Web push	Delivered to the device **whether or not the portal is open**. Arrives on the lock screen like any other notification, and opens straight to the event when tapped
Email	Full alert content with the same deep link, for the record and for recipients who prefer it

Each alert carries date, time, **track (up/down)**, **rail (up/down)**, **kilometrage**, and a **direct hyperlink to the event**. For standing-water conditions it carries a **one-click PTZ camera link**, on screen, in the push notification and in the email.

Meeting the five-minute obligation

The logger's default transmission interval is five minutes, which would consume the entire alert budget on its own. It does not, because your specification also provides an **immediate event-triggered transmission on any threshold breach** — so a breach is sent the moment it occurs rather than waiting for the next batch.

Our budget is therefore measured from the arrival of that event message: parse, evaluate, raise and dispatch, which is seconds rather than minutes. We will measure and report it end to end in Month 3 rather than assert it.

SMS is not included

At your direction SMS is out of scope, since web push reaches the user without the portal being open. This removes provider procurement, per-message cost and the associated contractual arrangements.

We will nonetheless build the delivery layer with **one channel interface behind which each transport sits**. Adding SMS later is then a new transport and a recipient setting — not a redesign of the alert engine.

One practical note on web push

Web push works on Chrome, Edge and Firefox on desktop and Android with no special step. **On iPhone and iPad it requires the portal to be added to the Home Screen once** (an iOS platform requirement, 16.4 and later); after that it behaves exactly like an app notification. It is a one-time, thirty-second step per device. We will cover it in the user guide and in training, and the portal will prompt for it on first visit from an iOS device.

We raise it now rather than at go-live because on-call staff are the people who depend on this channel most.

11. The three months

Three months from kickoff, twelve working weeks. **Each month ends with a release you can use**, not a progress report — so the system is in front of its users from Month 1 and shaped by their feedback while there is still time to act on it.

That is only possible because we are not starting from an empty repository. We already operate a production platform for Observator that ingests weather-station files over SFTP, stores and aggregates them, evaluates alert rules and serves a responsive Next.js portal. The M1 system is a substantial extension of that platform — the pump supervision, the vigilance logic, the flood state machine and the corridor map are new — but the ingestion, storage, aggregation, authentication and charting foundations already exist and are proven in service.

What makes three months work. The schedule has no slack for waiting on answers, so the items in §13 marked week 1 are genuinely week 1. Give us the logger's real output format and the confirmed thresholds on time and this plan holds; a fortnight's delay on either moves the end date by the same fortnight. We would rather say that now than discover it in Month 3.

Month 1 — Decisions, ingest and data

Release 1: readings from all eight loggers, arriving and visible.

Month 1 opens with **asking questions and writing the answers down**. Most of what goes wrong in a system like this is decided in the first fortnight and discovered in the last — a retention rule chosen carelessly, an alert path nobody agreed on, a data format assumed rather than seen.

Topics we will close out together in the first week:

- **Data**: exact logger output format, transmission method, and what a "complete" message looks like.
- **Retention**: confirmation of full detail for the contract term (see §4).
- **Alerting**: every threshold, dwell timer, vigilance period and the exact wording — including the values currently marked "pending confirmation".

- **Failure behaviour**: what the system should do when a sensor, a link or the database is unavailable, and what it must never do.
- **Security and residency**: AWS region, data residency, any MTS cyber standard to be assessed against, and MFA policy.
- **Roles**: who may see what, who may acknowledge, and who may operate a pump.
- **Integration**: PTZ camera access and the pump panel interface.

Week	Deliverable
1	Kickoff; **architecture workshop — open questions raised and answered**; decisions recorded with rationale; AWS accounts, environments, CI/CD; security baseline
2	Lightsail SFTP box hardened: per-site chroot, key-only auth, no inbound port; ingest agent with stability gates, atomic staging, retry with backoff and permanent archive
3	Rain, level, float, temperature, humidity, wind and battery/solar telemetry all parsed and stored; per-station configuration, units, calibration offsets
4	Time-series schema, indexes, rolling minute/hour/day aggregates; **failure drills** — kill the API mid-batch, take the database offline, disconnect the box, and prove nothing is lost. **Release 1**

Month 2 — Portal and alerting

Release 2: the corridor map, station screens, and alerts arriving on a phone.

Week	Deliverable
1	Portal shell, authentication, RBAC, station scoping, MFA; responsive layout down to phone width
2	**Corridor map** with live status and click-through; station detail and trends with thresholds drawn
3	Configurable threshold and wording table — every value a setting, not code; wind and temperature logic with dwell timers and rising/falling hysteresis; system-fault detection
4	Delivery: on-screen, **web push**, email; PTZ links in all three; alert lifecycle, acknowledgement and audit. **Release 2**

Month 3 — Rainfall, flood, pumps and handover

Release 3: the complete system.

Week	Deliverable
1	Rainfall rolling 1 h / 3 h / 3-day accumulation; vigilance timers (6/12/48 h) with restart-on-new-rain and automatic all-clear, surviving restarts; redundancy fallback with automatic re-wording
2	Flood state machine with staged reinstatement; **Marrickville pump screen** — **AUTO/MANUAL and supervised START/STOP**, role-gated and fully audited; flood-event screen with pump markers and event log
3	System-health dashboard (≤10-minute refresh); historical query engine with paged results and CSV export; administration screens for users, roles, thresholds and recipients
4	**Load test** including burst replay of a multi-day backlog; **security review and penetration test**; backup and **restore rehearsal**; end-to-end alert timing measured against the 5-minute obligation; documentation, runbooks, training, SAT support, handover. **Release 3**

12. Requirement traceability

Every item in the Statement of Requirements, mapped to the month that delivers it. Nothing is left unassigned.

Requirement	Delivered in
Rainfall measurement, accumulation windows, thresholds	M1, M3
Rainfall vigilance timers (6/12/48 h) and all-clear	**M3**
Flood level, float switch verification, staged reinstatement	M1, M3
Temperature thresholds with dwell timers	M2
Wind thresholds and TSR/block-line wording	M2

Requirement	Delivered in
Pump status, alarms and leading indicators	M2, M3
Supervised manual pump control	**M3**
Configurable threshold and wording table	M2
Redundancy and fallback with re-wording	**M3**
Screen, web push and email within 5 minutes	**M2**, measured M3
System-fault alerts within 30 minutes	M2
Alert content: date, time, track, rail, kilometrage, hyperlink	M2
PTZ camera verification links	**M2**
Alert and notification history, acknowledgement, export	M2, M3
Corridor map with click-through detail	**M2**
Trends and per-station screens	M2
Flood-event screen	M3
System health, ≤10-minute refresh	M3
Historical query engine and CSV export	M3
20+ named logins, RBAC, six standard roles, station scoping	M2
Custom roles, multiple roles per user, least privilege	M2
User administration — add, edit, suspend, remove	M2
Named accountability and permission re-check on every action	M2
MFA	M2
Audit trail	M2
Responsive HMI, phone to desktop	M2
Mobile access with push notification	**M2**
Solar/battery telemetry displayed and alarmed	M1, M3
Data transmission security	M1
No data loss on any failure	**M1**
Availability and planned-maintenance obligations	M3
Performance at scale	M1, M3
Backup, restore and failover	M3
Documentation, runbooks, training	M3

13. What we need from you

Three months is a short programme with no slack for waiting, so these dates are firm rather than indicative. Development can start without them, but each becomes blocking at the point shown.

#	Needed	By
1	**The logger's output format** — a real sample file per sensor type, and confirmation of the transmission method	**M1, week 1**
2	AWS region and any **data-residency** requirement	**M1, week 1**
3	Any MTS **cyber-security standard** the system must be assessed against	**M1, week 1**
4	Retention confirmation and the data-handover format at contract end	M1
5	Confirmed threshold values and exact alert wording currently marked "pending confirmation"	**M2, week 1**
6	**Kilometrage, track and rail** designations per station, and GPS positions	M2
7	The 20 portal users, their roles, and their station scope	M2
8	Notification recipient list, and who receives which severity	M2
9	**PTZ camera URLs** plus any authentication and network access needed to reach them	M2
10	Pump panel interface details — remote start/stop demand, run and fault status	**M3, week 1**

14. Assumptions

- The logger performs its own real-time pump control locally; the server supervises, records and alerts, and offers manual override. This is what makes a batched server interval appropriate.

- The logger logs 1-minute data and transmits in batches, with immediate transmission on a threshold breach. If a faster interval is later required, the ingestion path accommodates it without redesign; only the storage sizing changes.
- Sensors, loggers, masts, power, installation, EMC compliance and calibration are supplied and certified by Observator. Our scope is the software platform.
- Cellular connectivity to each station is provided and maintained by Observator.
- The trackside pump control panel accepts a remote start/stop demand and provides run and fault status.
- Cloud infrastructure is billed to Observator or MTS directly; our estimates exclude hosting fees.
- Notification recipients use a browser supporting web push, and iOS users add the portal to their Home Screen once (see §10).

15. Ways of working

- **Monthly releases** — every month ends with a version deployed for MTS to use, not a document describing progress.
- **Two-week sprints** inside each month, with a working demonstration at the end of each.
- **Fortnightly progress reports** and a review at each release.
- **Automated tests** on every change: unit, integration and end-to-end.
- **Every alert path tested against real recorded data**, not synthetic input.
- **Code review on every change**, with security-sensitive changes reviewed twice.
- **Documentation written as we go**, not assembled at the end.

16. Acceptance criteria

The system is complete when, demonstrably:

1. Every reading from every logger reaches the database, proven by a failure drill that takes the API and the database offline mid-flight and reconnects them.
2. A threshold breach produces screen, web push and email alerts **within 5 minutes**, measured end to end from the event-triggered transmission.
3. A web push notification arrives on a phone with the portal closed and the device locked.
4. Rainfall vigilance holds, restarts and clears correctly across a simulated multi-day event.
5. A sensor failure moves alerting to its fallback source and re-words the alert automatically.
6. An authorised operator can start and stop the Marrickville pumps from the portal, on desktop and phone, with the action recorded against their name.
7. Years of history can be queried and charted without degradation.
8. Twenty users can work concurrently with role and station scoping enforced.
9. A penetration test is passed with no unresolved high-severity findings.
10. A backup has been **restored** and verified, not merely taken.

17. In short

- **3 months**, three deliverables, nothing in the Statement of Requirements left unassigned.
- **A release every month** — the system is in front of its users from Month 1, not demonstrated at the end.
- **Month 1 is questions and answers** as well as code, and the answers are needed in week 1 for the schedule to hold.
- **One responsive application** for desktop and phone. No separate app to build, submit or maintain.

- **Web push** delivers alerts with the browser closed, so SMS is not required — with the delivery layer built so SMS can be added later without redesign.
- **Two components, not four.** The queue and the cache went with the one-second requirement; three independent buffers already protect the data, and the file on disk outlasts any queue's retention.
- **Nothing is ever lost and nothing is ever deleted** — the standard the whole ingestion design is built to.